Architecture

Project: Auditable AI Agents

---

1. Layers

```
+-------------------------------------------------------------+
| app/            (side effects: files, network, processes)   |
|   run_demo.py   chain_client.py   logstore.py               |
|   ollama_llm.py anthropic_llm.py                            |
+-------------------------------------------------------------+
| domain/         (pure functions, immutable data)            |
|   agent.py   chain.py   merkle.py   audit.py                |
+-------------------------------------------------------------+
| core/           (primitives)                                |
|   hashing.py                                                |
+-------------------------------------------------------------+
| contracts/AuditAnchor.sol      (runs on the blockchain)     |
+-------------------------------------------------------------+
```

Rule: `domain` and `core` never read files, call the network or print. This keeps them easy to test and means the cryptographic logic is the same whichever model or chain is used.

---

2. Data flow

1. **Run.** `run_agent(task, llm, tools, log, clock)` loops: ask the model, run the tool it chose, append every event to the log. The log is an immutable tuple that is passed along and returned, never mutated.
2. **Save.** `save_log` writes the entries as JSON Lines (the off-chain copy).
3. **Anchor.** `anchor_pending` asks the contract how many entries are already anchored, plans new batches, builds each batch's Merkle root and sends `anchor(root, count)`.
4. **Verify.** `verify_against_chain` loads the saved log and the batches from the contract, then checks the hash chain and each batch root.

---

3. What each check catches

Attack	Hash chain check	On-chain root check
Edit entry content	Caught, exact entry	Not needed
Delete / insert / reorder	Caught, at or near the change	Caught
Truncate the end	Not caught (a shorter chain is still a valid chain)	Caught (log shorter than what was anchored)
Rewrite an entry and recompute all later hashes	Not caught (chain is self-consistent)	Caught (batch root differs); location is batch-level

Attack	Hash chain check	On-chain root check
Add entries after the last anchor	Caught only if they break the chain; they are reported as "not yet anchored"	Not covered until anchored

The last row is the delay window measured in Experiment 2: entries are only protected once their batch is anchored.

---

4. The model interface

Every model is an `llm(messages) -> action` function, where an action is either `{"type": "tool_call", "tool", "args"}` or `{"type": "final", "content"}`, optionally with a `meta` dictionary (model name, token counts) that is logged. Implementations:

Function	Where	Needs
scripted_llm	domain/agent.py	nothing (deterministic, for tests and offline demos)
make_ollama_llm	app/ollama_llm.py	local Ollama server
make_anthropic_llm	app/anthropic_llm.py	ANTHROPIC_API_KEY

---

5. Contract and Python must agree

Rule	Python	Solidity
Hash	hashlib.sha256	sha256()
Leaf	sha256(0x00 + entry_hash)	sha256(abi.encodePacked(0x00, entryHash))
Inner node	sha256(0x01 + left + right)	sha256(abi.encodePacked(0x01, left, right))
Odd node	carried up	not applicable (proof simply has no step)

`tests/vectors/audit_vectors.json` is generated by Python and checked by the Hardhat tests.

---

6. Known design limits

- One account per agent identity; no key management or rotation.
- The logger and the agent run in the same process, so a compromised host can write false entries before anchoring.
- Batch-level (not entry-level) localisation after a consistent rewrite.
- Local chain only; no testnet or production deployment.